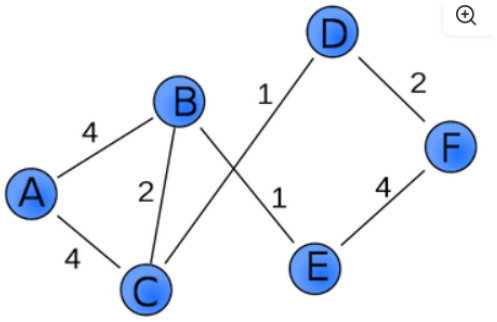
Artificial Intelligence Lab

Dibyasundar Das

13 August 2026

LAB 03



- Translate the visual graph above into Python code using two different data structures. Assume the nodes are labeled A, B, C, D, E, F.
  1. **Adjacency List (using a Dictionary)** Create a variable named `adj_list` where each key is a node and the value is a list of all nodes it is directly connected to.
  2. **Adjacency Matrix (using a Nested List)** Create a variable named `adj_matrix` which is a 2D list (list of lists) representing the connections. Use `1` for a connection and `0` for no connection.
  3. Write python code to Plot graphs in both cases.

> ⚠ **Note**
>
> You must pick a sentinel `math.inf` for "no edge." Explain why `0` is wrong here.

- Implement three searches from scratch. Each should return `(path, total_cost, visit_order)`.
  1. `bfs_path(adj_list, 'A', 'F')`. **Ignore weights during the search**; compute the cost of the returned path afterwards.
  2. `dfs_path(adj_list, 'A', 'F')` **Ignore weights during the search**; compute the cost of the returned path afterwards.
  3. `dijkstra_path(adj_list, 'A', 'F')` — `heapq`, with `dist` and `parent` dictionaries.
     - Print one table: with algorithm name, path, edge count , total cost.
- Visualise and compare three figures, showing returned path per algorithm in red.